



GLC2D Game Library를 사용한 2D Game 제작

Heesung Oh

<https://github.com/3dapi>



- Software Developing Flow
- Introduction to Game Components
- GLC2D Game Library
 - ◆ glc2d System
 - ◆ glc2d Texture
 - ◆ glc2d Input
 - ◆ glc2d Font
 - ◆ glc2d Sound



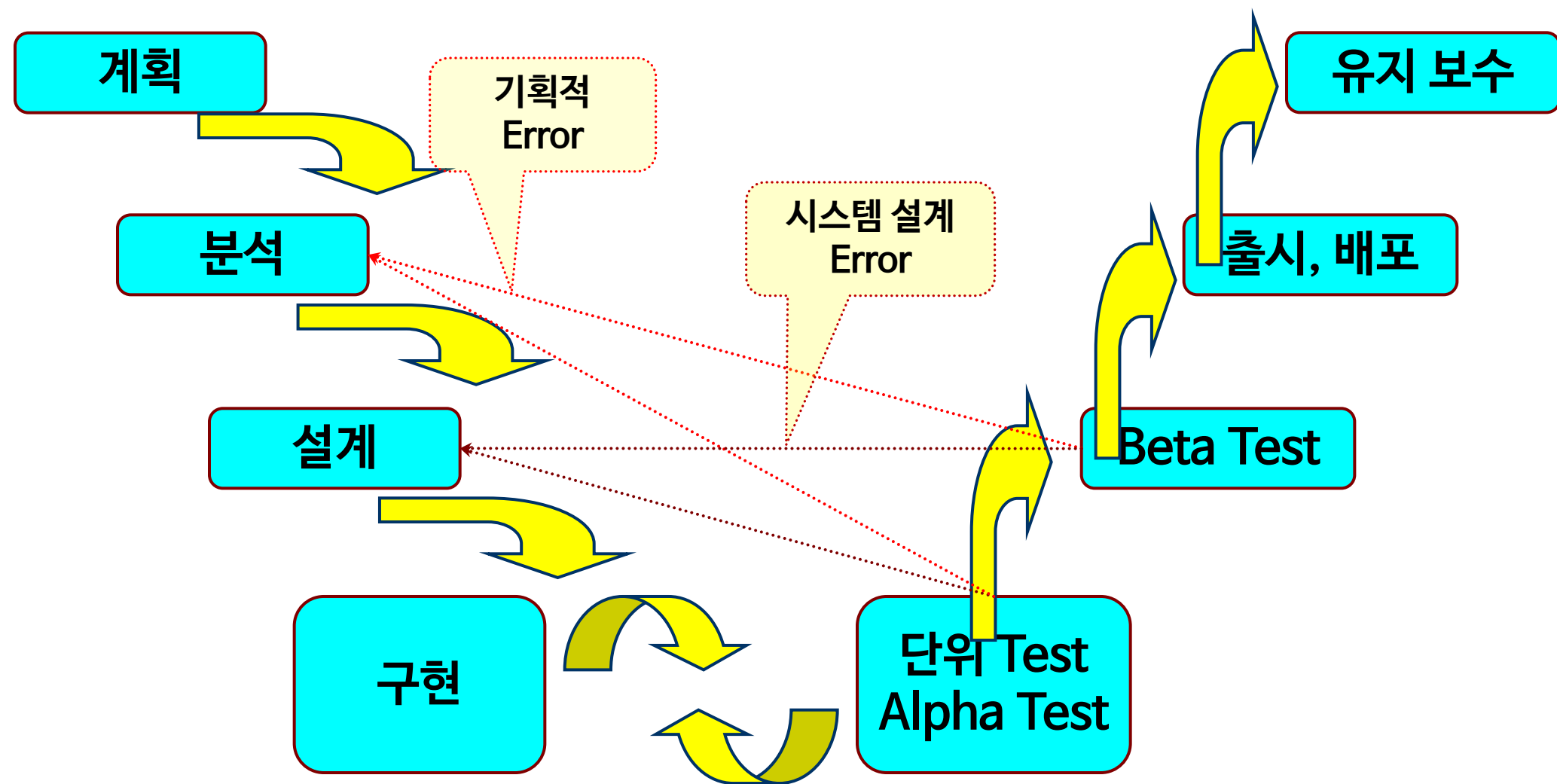


PART 1. Software Developing Flow





1.2 Software Developing Flow





● 계획

- ◆ 개발 내용 계획: 기술적인 이슈와 개발 업무 계획
- ◆ 비용 계획: 시간, 인원 등의 소요 비용 산정
- ◆ 조직 계획: 개발 팀 구성 및 역할 배정
- ◆ 제안서 작성

● 분석

- ◆ Domain 분석: 객체 정의 → 관계 설정 → 추상화
- ◆ Play 분석: 실행 흐름 작성
- ◆ 그래픽 Concept → Game 데이터 설정
- ◆ 상세 기획 작성



- 게임 시스템, 자료구조 설계
 - ◆ 공통 모듈: 툴, 콘텐츠 등에서 공통으로 사용할 모듈
 - ◆ 렌더링 머신
 - ◆ 장면 관리자
 - ◆ 네트워크 입출력
 - ◆ Database 설계
 - ◆ 입력장치, 미디어 플레이 인터페이스 설계
- 게임 엔진 설계
 - ◆ 지형 엔진, 물리 엔진
 - ◆ 그래픽 플러그인, 캐릭터 애니메이션
 - ◆ GUI, 스크립트





- 툴 프로그램 구현
 - ◆ 리소스 관리 툴
 - ◆ 지형 툴
 - ◆ 캐릭터 툴
 - ◆ Effect, GUI 툴
- 콘텐츠 구현
 - ◆ 네트워크 입출력 데이터 형식 결정
 - ◆ 게임 플레이 위상 구현
 - ◆ 렌더링 pass 구현
 - ◆ AI 구현
 - ◆ 엔진 모듈 통합 → 콘텐츠 구현





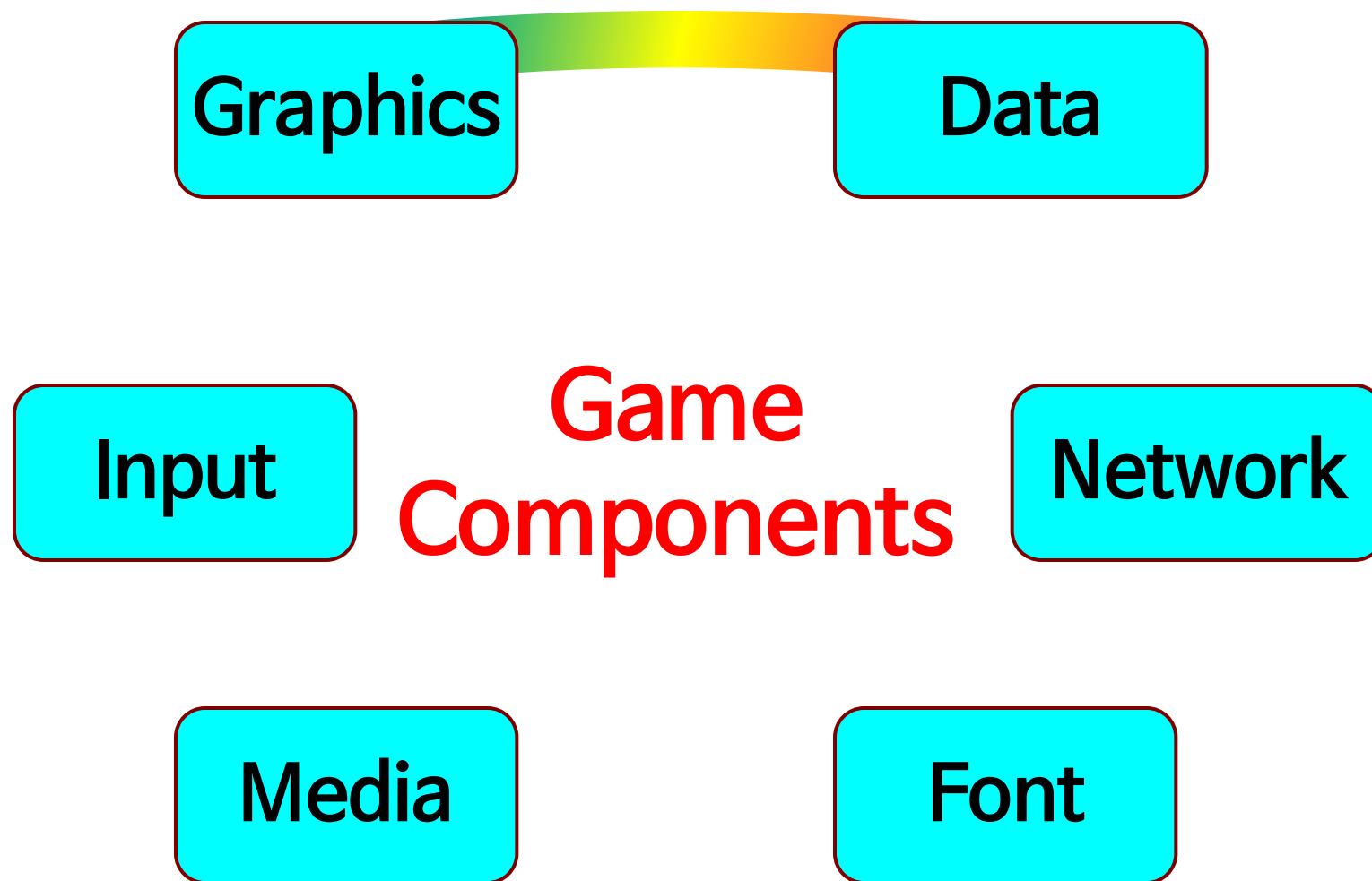
- 테스트
 - ◆ 기능(Play) 중심 알파 테스트
 - ◆ 그래픽 Quality, 성능 중심의 클라이언트 알파 테스트
 - ◆ 네트워크 I/O 중심의 클라이언트/서버 베타 테스트
- 배포, 유지보수
 - ◆ 버그 패치
 - ◆ 기능 강화
 - ◆ 보안 강화





PART 2. Game Components







- 2D
 - ◆ Image
 - ◆ Texture
 - ◆ 2D 모델
 - ◆ GUI(Graphic User Interface)
 - ◆ Library:
 - Video Buffer: 3D Rendering Pipe Line 이 없는 시스템
 - Direct3D 또는 OpenGL 등 3D Rendering Pipe Line 로 구현
- 3D
 - ◆ 3D Mesh
 - ◆ Transform & Lighting
 - ◆ Library: Direct3D, OpenGL





- Keyboard
 - ◆ Keyboard
 - ◆ Keypad, 버튼
 - ◆ Library: DirectInput, API 함수

- Mouse
 - ◆ Mouse
 - ◆ Multi touch screen
 - ◆ Library: DirectInput, API 함수

- Joystick
 - ◆ Library: DirectInput, API 함수





- Sound

- ◆ Analog signal → Digitize
- ◆ Mono
- ◆ Stereo
- ◆ 3D sound
- ◆ Library: DirectSound, OpenAL, OpenSL

- Video

- ◆ Library: DirectShow, API 함수

- Music

- ◆ Direct Music: 연주를 위한 악보 개념의 모듈이었으나 구현이 Sound로 통합





- 2D Font
 - ◆ 2D String 출력
 - ◆ Library: D3DX, API, FreeTypeFont
- 3D Font
 - ◆ String → Vertex Buffer 변환 → 3D 출력
 - ◆ Library: D3DX, API 함수





- I/O Model

- ◆ Windows: AsyncSelect, EventSelect, IOCP
- ◆ Linux, Unix: Epoll, Kqueue
- ◆ BSD: Select
- ◆ Library: Winsock, API

- Database

- ◆ 관계형 데이터 베이스
- ◆ 객체지향형 데이터 베이스
- ◆ XML
- ◆ 종류: Oracle, MSSQL, MYSQL, Embedded용 SQL
- ◆ Library: ODBC, OLEDB, API





- Distributed Server Model

- ◆ 기능 분산

- 패치, 인증, 콘텐츠 서버

- ◆ 데이터 분산

- Zone, Room, DB 서버

- ◆ 공간 분산

- Server Group(서버 군) 구성





- **Game Play Data**
 - ◆ Game Play와 직접 연관있는 데이터
 - ◆ 반드시 실행
 - ◆ 네트워크 입출력 데이터, Game Level, UI
- **Rendering Data**
 - ◆ 연출 데이터
 - ◆ 필요에 따라 실행이 안될 수 있음
 - ◆ Particle,





PART 3. GLC2D Game Library



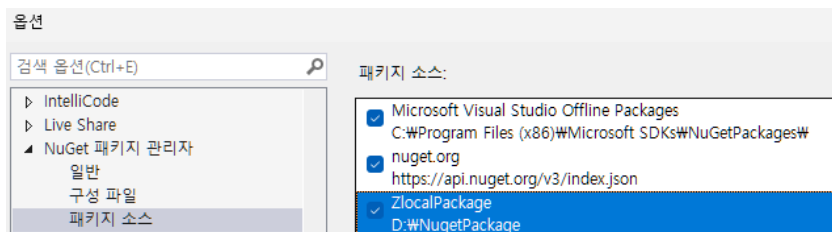


- Window 생성/ 해제
 - ◆ glc2d_CreateWin
 - ◆ glc2d_DestroyWin
- 프로그램 실행 (Run-while)
 - ◆ glc2d_Run
- Ex01_start 예제 참조

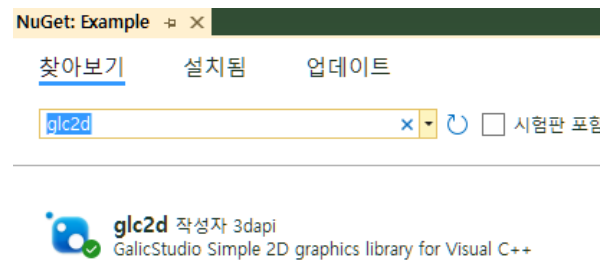
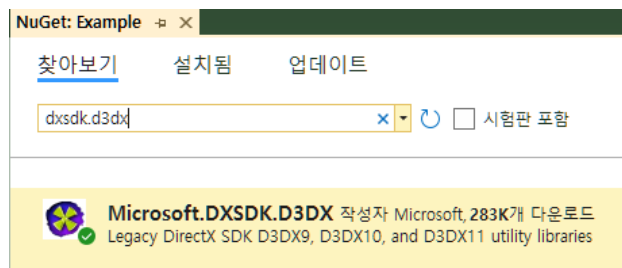




- C 또는 D drive 폴더 생성
 - ◆ Ex) D:\WNugetPackage
- Visual studio Package 관리
 - ◆ 메뉴 → 도구 → 옵션 → Nuget 패키지 관리자 → 패키지 소스 생성한 폴더 등록



- glc2d.xxxx.nupkg 다운로드 → NugetPackage 로 이동
 - ◆ https://github.com/3dapi/bs17_2d_lib/tree/master/gls2d_v0.1/nuget
- dxsdk.d3dx,glc2d 설치





- 시스템 내부에서 Update/Render 반복
 - ◆ 사용자가 정의한 Update/Render 함수를 전달해야 프로그램이 완성됨
- Rendering 함수 설정
 - ◆ glc2d_SetRender
 - ◆ 형식: int 함수_이름(void)
- FrameMove 함수 설정
 - ◆ glc2d_SetFrameMove
 - ◆ 형식: int 함수_이름(void)





- **텍스처 생성**
 - ◆ glc2d_TextureLoad
 - ◆ 텍스처가 생성되면 순차적으로 인덱스가 반환
 - ◆ 실패하면 -1 반환
- **텍스처 해제**
 - ◆ glc2d_TextureRelease
 - ◆ 강제로 해제하는 경우를 제외하고 자동으로 해제 됨
- **텍스처 너비**
 - ◆ glc2d_TextureWidth
- **텍스처 높이**
 - ◆ glc2d_TextureHeight



- **텍스처 그리기**

- ◆ **glc2d_Draw2D**

- ◆ **인수:**

- **텍스처 인덱스**
 - **원본 이미지 영역: (좌-상단, 우-하단), NULL이면 전부 렌더링**
 - **화면에서의 위치 x, y: NULL 이면 (0,0) 위치에 렌더링**
 - **크기 비율 x, y: NULL이면 (1.0, 1.0)**
 - **회전 중심 위치 x, y: NULL 이면 (0,0)**
 - **회전 각: Radian**
 - **색상 곱셈 값: 32bit ARGB로 구성**
 - **단색화(Monotone): TRUE/FALSE**

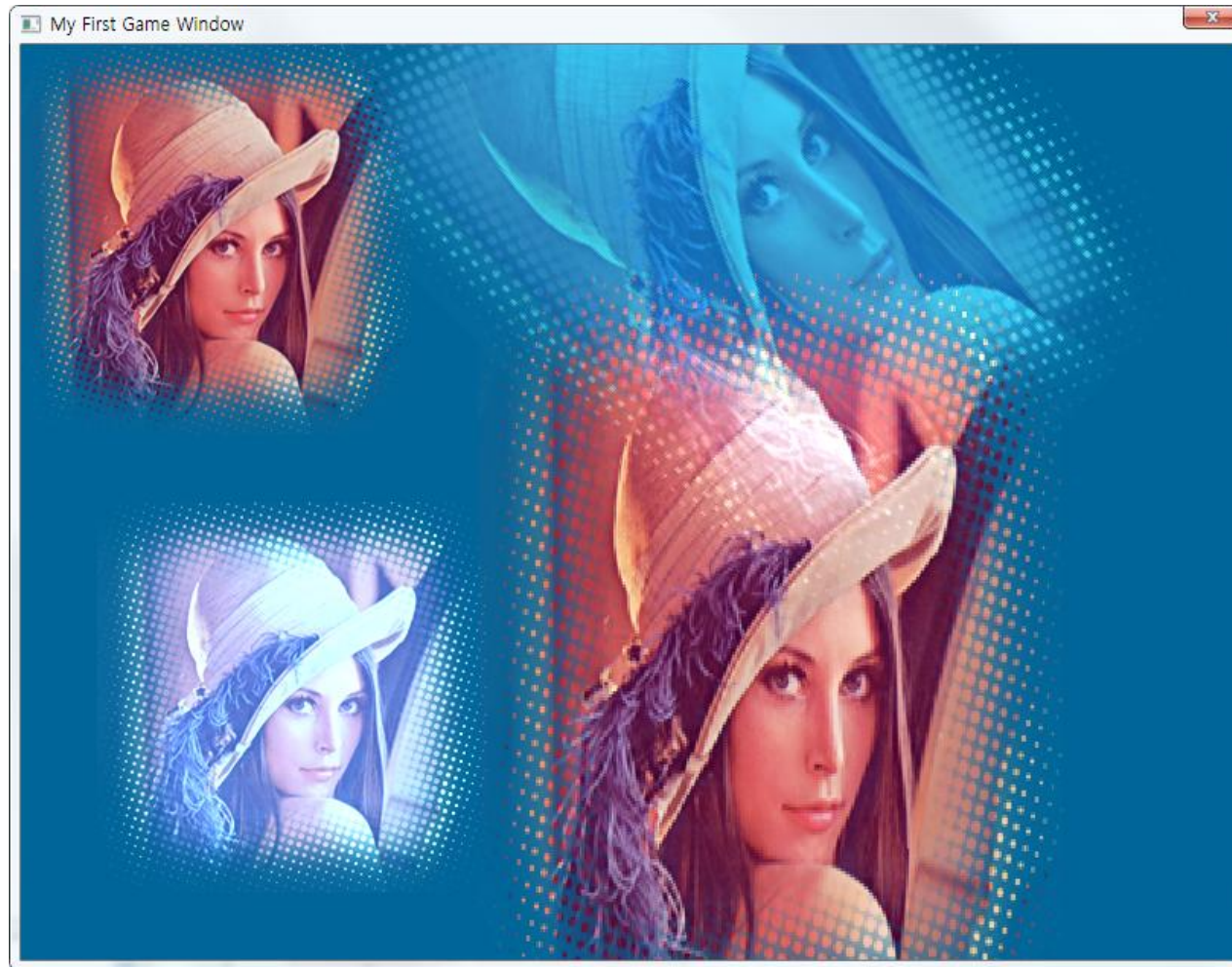
- ◆ **색상은 32bit 8-ALPHA, 8-RED, 8-GREEN, 8-BLUE로 처리함**

- **알파 블렌딩 옵션**

- ◆ **glc2d_DrawAlphaOption**

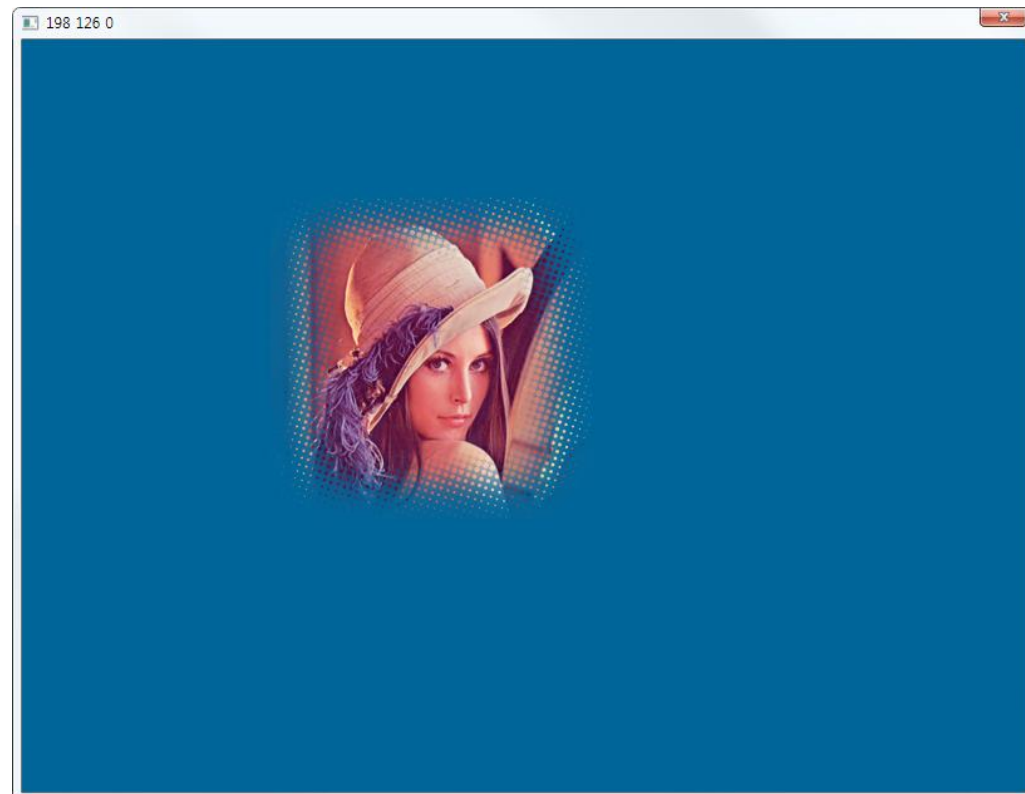


- 텍스처 그리기 : Ex02_Texture 예제

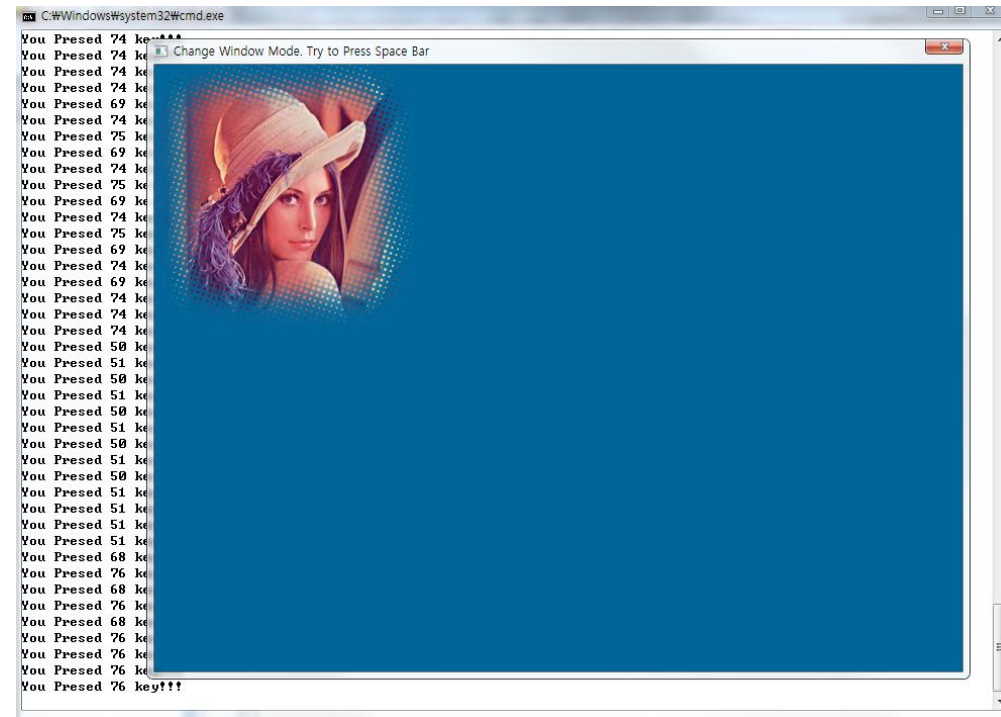


- Mouse 위치
 - ◆ X: glc2d_GetMouseX
 - ◆ Y: glc2d_GetMouseY
 - ◆ Z(Wheel): glc2d_GetMouseZ

- 이벤트
 - ◆ glc2d_GetMouseEvent
 - ◆ 인덱스
 - L button: 0
 - R button: 1
 - M button: 2

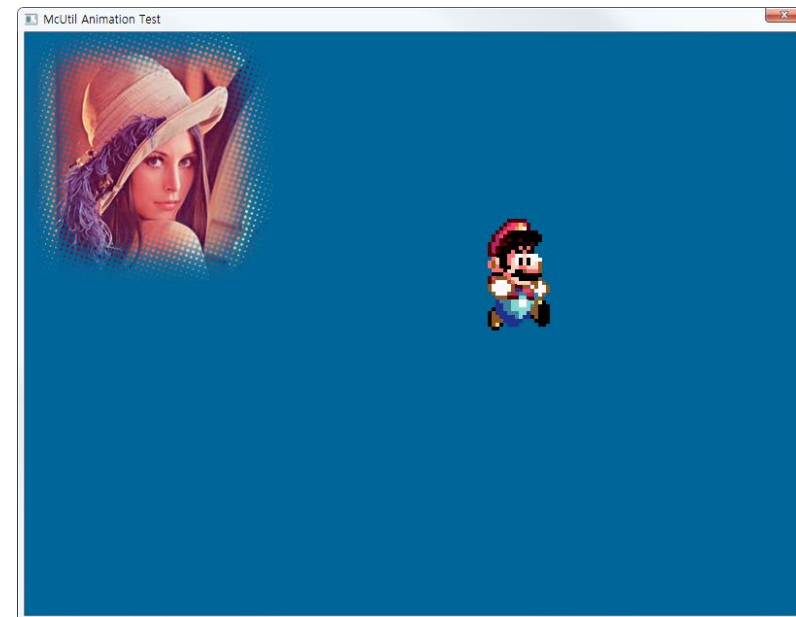


- Keyboard Event:
 - ◆ glc2d_GetKeyboard
 - ◆ 256 unsigned char 배열 사용
- Mouse, Keyboard 이벤트
 - ◆ NONE : 0
 - ◆ DOWN(One-Click): 1
 - ◆ UP : 2
 - ◆ Press : 3
 - ◆ Double Click : 4





- Animation
 - ◆ 시간에 대한 이미지 영역을 설정
- 시간(milliseconds)
 - ◆ glc2d_TimeGetTime
- Simple Animation 순서
 - ◆ $\text{Frame Index} = \text{Time} / \text{Ani_Speed}$
 - ◆ $\text{Frame Index} \% = \text{Max Frame}$





- 폰트 생성

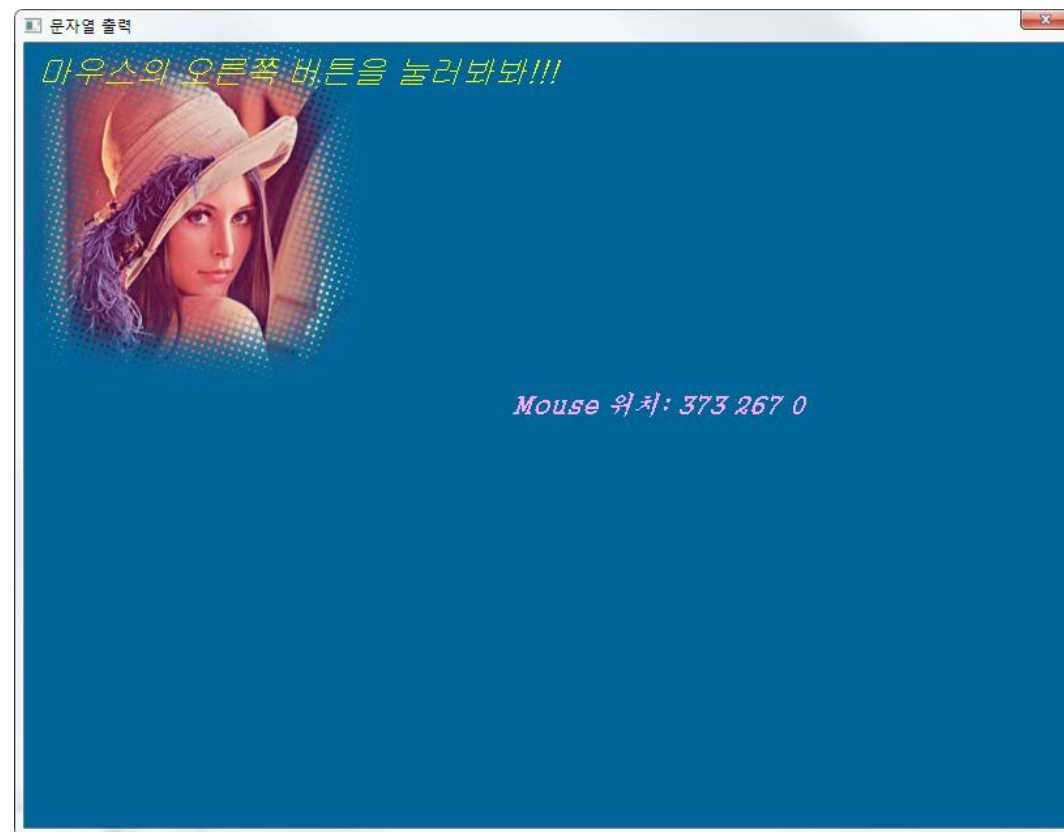
- ◆ glc2d_FontCreate

- 출력

- ◆ glc2d_FontDrawText

- ◆ 인수:

- 폰트 인덱스
- 좌-상단, 우-하단 영역
- 색상: 32bit ARGB
- Format
- Variables





- 사운드 생성/해제: `glc2d_SoundLoad /Release`
 - ◆ wav 파일만 지원
- 재생: `glc2d_SoundPlay`
- 멈춤: `glc2d_SoundStop`
- Reset: `glc2d_SoundReset`
 - ◆ 처음 위치로 옮김
- 실행 중인지 확인: `glc2d_SoundsIsPlaying`





- 윈도우 핸들: `glc2d_GetHwnd`
- 윈도우 크기, 스케일: `glc2d_GetScn{W|H|Scale}`
- 윈도우 스타일: `glc2d_{Set|Get}WindowStyle`
- Icon 설정: `glc2d_SetWindowIcon`
- Clear 색상 설정: `glc2d_{Set|Get}ClearColor`
- 상태 보이기: `glc2d_SetStateShow`
- 마우스 숨기기: `glc2d_SetCursorShow`

